GAMECA v0.4.39

Installation, Quick-Start & Output Reference

Anmol Dash

June 27, 2026

Contents

1 What is GAMECA?

GAMECA (Gene Alignment, Motif, Expression and Clustering Analysis) is a native macOS desktop application for the comprehensive downstream characterisation of transposable-element (TE) families. It ships as a `.dmg` installer containing a Tauri/React frontend and a bundled Python analysis backend. Starting from nothing more than a RepeatMasker family name and a genome assembly code, GAMECA runs the following stages:

1. **Data prep** — fetch all annotated genomic copies from UCSC RepeatMasker or Dfam, then extract sequences (from a local genome FASTA or via the UCSC API).
2. **Clustering** — group copies by k-mer profile (SVD $\rightarrow$ UMAP/t-SNE $\rightarrow$ HDBSCAN).
3. **Alignment** — per-cluster MAFFT alignment, CIAlign cleaning, and majority-vote consensus sequences.
4. **Motif analysis** — scan every copy for JASPAR transcription-factor binding motifs.
5. **Enrichment** — Fisher's exact test for motifs over-represented in each cluster.
6. **TFBS analysis** — count and visualise transcription-factor binding sites per cluster and per strand.
7. **GO annotation** — map enriched motifs to their transcription factors and Gene Ontology terms.
8. **Expression** — per-cluster expression boxplots and differential-expression tests (when an expression matrix is provided).
9. **Primer design** — family- and cluster-specific PCR primers with genome-wide specificity checking.
10. **Stage 11 standout analyses** — phylogenetics, CRISPR gRNA off-target scoring, $3'$ transduction detection, antisense promoter scanning, CTCF/TAD overlap, epigenetic overlay, ortholog insertion presence, multi-assembly liftover, ColabFold consensus ORF structure prediction, repeat landscape, LTR structural classification, subfamily resolution, and differential expression between clusters.

Analyses run either **locally** on your Mac or are submitted as independent **batch jobs** on an LSF or Slurm HPC cluster over SSH. No terminal knowledge is required for typical use.

2 System requirements

Component	Requirement
macOS	12 Monterey or later (Apple Silicon *or* Intel)
Disk	∼3 GB (app + Python environment + first ColabFold model weights)
RAM	8 GB minimum; 16 GB or more recommended for large families
Internet	Required for UCSC/Dfam sequence fetching, first-run package install, JASPAR/GO lookups, and HPC SSH
HPC cluster	Optional; LSF (`bsub`) or Slurm (`sbatch`) reachable by SSH from your Mac

3 Download

Installers are published on the GitHub Releases page:

https://github.com/anmol-dash/te-scraper-and-analysis/releases

Download the `.dmg` that matches your Mac's processor:

Architecture	File to download
Apple Silicon (M1/M2/M3/M4)	`GAMECA_0.4.39_aarch64.dmg`
Intel	`GAMECA_0.4.39_x64.dmg`

If unsure which processor your Mac has, click the Apple menu → **About This Mac** and read the "Chip" or "Processor" line.

4 Installation

1. Double-click the downloaded `.dmg` to mount it.
2. Drag **GAMECA.app** into your **Applications** folder.
3. Eject the disk image (`Cmd-E`).
4. **First launch only — Gatekeeper:** because GAMECA is distributed outside the Mac App Store, macOS blocks the first open. **Right-click** (or Control-click) **GAMECA.app** → **Open** → **Open**. This prompt appears once only.

5 First launch and automatic setup

On first launch GAMECA performs a one-time setup (∼2–3 minutes):

1. The bundled Python sidecar starts and sends a `ping` handshake to the frontend; a spinner shows until it completes.
2. The app checks for a Python virtual environment at `$HOME/gameca_venv`. If absent, it creates one and installs all dependencies from the bundled `requirements.txt` (NumPy,

pandas, scikit-learn, UMAP, HDBSCAN, BioPython, MAFFT bindings, CIAlign, MOODS, `colabfold[alphafold]`, and more).

3. A progress bar and log stream in the **Log** panel. Do not close the app during this step.
4. When the status indicator turns green the app is ready.

Note: ColabFold model weights (∼4 GB) are *not* downloaded here; they are fetched the first time a structure-prediction job runs and cached in `$HOME/.cache/colabfold`.

6 The interface

Sidebar (left). Every pipeline step, grouped into *Data prep*, *Core analysis*, *Enrichment*, *Primer design*, *Full pipeline*, and *Results*. Click a step to open its configuration panel.

Configuration panel (centre). Parameters for the selected step: family name, assembly, output directory, queue, and step-specific options. Fill in and click **Run on Cluster** or **Run locally**.

Log panel (bottom). Streams live output line-by-line: stage checkpoints, progress bars, and errors.

7 Connecting to an HPC cluster

1. Click **Settings** (gear icon) → **HPC Connection**.
2. Enter **Hostname**, **Username**, **SSH key path** (blank to use the SSH agent), **Scheduler** (`LSF`/`Slurm`/*Auto*), **Queue / partition**, and a writable **Work directory** (e.g. `/home/amodz/anmol/gameca`)
3. Click **Test connection**. A green tick confirms the scheduler was detected and the work directory is writable.

Duo / two-factor clusters: if your cluster requires MFA on every login, add a `ControlMaster` entry to `~/.ssh/config` so the app reuses one authenticated session:

```
Host hpclogin1.university.edu
    ControlMaster auto
    ControlPath ~/.ssh/cm-%r@%h:%p
    ControlPersist 8h
```

8 Running an analysis

8.1 Check the locus count first

Data prep → RMSK locus count (or **Dfam locus count**) verifies the family name and reports how many copies exist before you commit to a full job.

8.2 Single family — on the cluster

1. **Full pipeline → Batch job**.
2. Set **Family** (e.g. `THE1A`), **Assembly** (`hg38`/`hg19`/`mm10`/`mm39`), **Output directory**, and optionally **Notify email**.

3. Click **Run on Cluster**. GAMECA uploads the scripts, writes a `bsub`/`sbatch` job, and submits it. The job ID prints in the Log panel.
4. Monitor in **Results** → **File browser** or with `bjobs -w` on the cluster.
5. When done, **Results** → **Retrieve** `rsync`s the output folder back to your Mac.

8.3 Single family — locally

Full pipeline → **Batch job**, fill in **Family** and **Assembly**, then **Run locally**. Output lands in `$HOME/gameca_results/<family>/`. Suitable for families up to ∼5,000 copies on 16 GB RAM; larger families and ColabFold prediction should run on HPC.

8.4 Many families — batch submission

Submit many families as independent jobs with the bundled `submit_diego_families.py`. Each family produces *two* separate LSF jobs — a main job (core pipeline + Stage 11 except ColabFold) and a dependent fold job (ColabFold).

```
source ~/gameca_venv/bin/activate

python submit_diego_families.py \
    --outdir /home/$USER/gameca_results \
    --queue normal \
    --notify-email you@institution.edu

python submit_diego_families.py --dry-run # preview without submitting
```

Both jobs per family appear immediately in `bjobs`: `gam_<FAMILY>_main` starts when a slot frees; `gam_<FAMILY>_fold` stays PEND until its main job finishes, then starts automatically. Two emails are sent per family (one when the main job ends, one when the fold job ends).

9 Output reference

A run writes one folder per family. The tree below is the complete layout; the subsections that follow show *what is inside each file* so you can read the results without ever opening the app.

```
<outdir>/<family>/ e.g. gameca_results/the1a/
  01_data/
    the1a_clustered.csv master table: every copy + cluster
  02_statistics/
    cluster_summary.csv one row per cluster (size, GC, length)
    per_cluster/cluster_<c>_statistics.txt
  03_clustering/
    clustering_visualization.html interactive UMAP scatter
    clustering_visualization_expr.html (if expression provided)
  04_alignments/
    alignment_stats.txt
  05_consensus/
    the1a_consensus.fa one consensus per cluster + global
  05_tfbs/
    cluster_tf_binding_counts.csv TF binding sites per cluster
    strand_tf_binding_counts.csv
```

```
  cluster_tf_binding_heatmap.html
  tf_binding_top_overlaps.html
06_primers/
  cluster_top5_primers.csv recommended primers per cluster
  selected_primers_summary.csv
07_visualizations/
  index.html dashboard linking every figure
cluster_alignments/
  cluster_<c>_seqs.fa / _aligned.fa / _consensus.fa
  all_cluster_consensuses.fa
cialign_plots/
  index.html open in a browser to view alignments
motif_analysis/
  all_overlaps.tsv every copy x motif overlap
  overall_motif_counts.csv motif frequency table
  overall_top_motifs.png
enrichment_results/
  cluster_<c>_enrichment.csv Fisher test per motif per cluster
  cluster_<c>_enrichment_annotated.csv
  enrichment_heatmap.png
go_annotations/
  gene_functions.csv TF -> gene -> GO lookups
  go_bp_top_terms.csv
  go_bp_top_terms.png
reports/ Stage 11 results (see 9.7)
  standout_analysis.log
  fig_*.pdf / *_per_copy.csv / *_report.txt / *_measured_values.tex
  fold/ ColabFold 3D structures
bsub_main.out / bsub_fold.out live cluster job logs
```

9.1 Clustering: the master table

`01_data/<family>_clustered.csv` is the central output: one row per genomic copy. `Cluster` is the group it was sorted into; `chr/start/stop/strand` give its location; `Seq` is its DNA; `umap_x/umap_y` are the 2-D coordinates plotted in `03_clustering/clustering_visualization.html`.

```
Seq,Cluster,chr,start,stop,strand,te_name,umap_x,umap_y
tgttattagacgcg...,3,chr1,3031358,3031710,-,IAPLTR1a_Mm,4.21,-1.08
aatattcagttgtc...,1,chr2,9881204,9881559,+,IAPLTR1a_Mm,-2.66,3.94
```

`02_statistics/cluster_summary.csv` summarises each cluster (number of copies, mean length, GC content). The interactive UMAP scatter in `03_clustering/` colours every copy by cluster — open it in a browser and hover for per-copy detail.

9.2 Alignment and consensus

`cluster_alignments/cluster_<c>_consensus.fa` is the "average" sequence of each cluster in FASTA format; the `>` line is the name and the lines below are the sequence. `all_cluster_consensuses.fa` bundles them all and is the input for downstream structure/divergence modules.

```
>IAP_cluster3_consensus
GGGGCTATTTTTGGTTGGTGTGGACGTTTTTTGTCTGGAAAATTAAGAGG
ATGCCAAAAGGTCCGATTTGAAAAAACAGTCTTTACATTGGAAATATCTA
```

cialign_plots/index.html is an HTML gallery of the CIAlign before/after/markup images — the quickest way to judge how clean each cluster's alignment is.

9.3 Motif analysis (motif_analysis/)

This stage scans every copy for JASPAR transcription-factor binding motifs. **all_overlaps.tsv** is the raw result: one line per (copy, motif) overlap, giving the copy coordinates, the motif name, its position, score, and strand — this is the table every downstream enrichment/TFBS step is built from.

overall_motif_counts.csv collapses that to a simple frequency table (how many times each motif was found across all copies), and **overall_top_motifs.png** plots the top 20.

```
Motif_name,count
CTCF,1840
SP1,1422
KLF4,1190
```

9.4 Enrichment (enrichment_results/)

Finding a motif is not the same as it being *characteristic* of a cluster. For each cluster, GAMECA runs a Fisher's exact test asking "is this motif over-represented in this cluster versus the rest of the family?" **cluster_<c>_enrichment.csv** holds one row per motif, sorted by `P_Value`:

```
Motif,In_Cluster,Total_With_Motif,Cluster_Size,Total_Loci,Odds_Ratio,P_Value
CTCF,118,402,140,1485,6.91,3.2e-41
SP1,77,388,140,1485,3.10,1.1e-12
```

- `In_Cluster` — copies in this cluster with the motif.
- `Total_With_Motif` — copies family-wide with the motif.
- `Odds_Ratio` — how much more frequent the motif is in this cluster (> 1 = enriched).
- `P_Value` — Fisher's exact significance; rows with `P_Value` below the threshold are the cluster's signature TFs.

cluster_<c>_enrichment_annotated.csv adds the transcription-factor name, a one-line summary, and GO terms (see §9.6) to every significant motif. **enrichment_heatmap.png** shows clusters $\times$ enriched motifs at a glance.

9.5 TFBS analysis (05_tfbs/)

Where enrichment asks "which motifs are statistically special", the TFBS stage simply *counts* binding sites so you can compare raw binding load across clusters. **cluster_tf_binding_counts.csv** has one row per (cluster, motif):

```
Cluster,Motif_name,Binding_Sites
0,CTCF,118
0,SP1,77
1,REST,54
```

strand_tf_binding_counts.csv breaks the same counts down by DNA strand. The two `.html` files are interactive Plotly views: `cluster_tf_binding_heatmap.html` (clusters × TFs, log-scaled) and `tf_binding_top_overlaps.html` (the 30 highest single overlaps). Open either in a browser.

9.6 GO annotation (go_annotations/)

This stage answers "what do the enriched transcription factors actually *do*?" Each significant motif is mapped to its gene via `mygene.info`, and **gene_functions.csv** records the function and Gene Ontology terms:

```
Gene,Symbol,Name,Summary,GO_BP,GO_MF,GO_CC
CTCF,CTCF,CCCTC-binding factor,"Chromatin insulator...",chromatin organization;...,DNA
    binding;...,nucleus;...
```

go_bp_top_terms.csv ranks the most common Biological Process terms across all enriched TFs, and `go_bp_top_terms.png` (plus per-cluster `cluster_<c>_go_bp.png`) plots them.

```
GO_Term,Count
regulation of transcription,42
chromatin organization,28
```

If no motifs pass the significance threshold the stage writes a `GO_SKIPPED.txt` note instead of empty files — expected for small or highly divergent families.

9.7 Primers (06_primers/)

cluster_top5_primers.csv lists the recommended PCR primers per cluster. `coverage` is how many copies the primer matches; `total_expr` sums their expression; `rows_covered` names the copies. `selected_primers_summary.csv` is the global shortlist with the selection `strategy` used.

```
cluster,primer,coverage,total_expr,rows_covered
0,CCTAGCTATGCTGCCTTG,6,1119.19,"8,9,10,11,12,13"
```

9.8 Stage 11 standout analyses (reports/)

Every Stage 11 analysis writes the *same four kinds of file*, so once you can read one you can read them all. Using phylogenetics (`phylo`) as the example:

File	What it is
`fig_phylo_tree.pdf`	The figure — open and look at it. Each analysis makes one or more `fig_*` PDFs/PNGs.
`phylo_per_copy.csv`	One row per copy, adding this analysis's columns (here: age, divergence, ORF status). Join back to the master table by coordinates or `TE_ID`.
`phylo_report.txt`	A plain-English summary of the headline numbers.
`phylo_measured_values.tex`	The same numbers as LaTeX macros for the PDF report; ignore unless compiling the paper.

The `*_report.txt` files need no software and are the fastest way to read a result:

```
GAMECA Phylogenetics / Divergence-Age --- Measured Values
  Family: IAP
  Copies analysed: 12577
  Median est. age: 423.18 Myr
  Intact (ORF>=thr): 3374
  -- Top master/source element --
  Name: chr8:4886476
```

Some analyses also write a specialised table, e.g. `grna_offtarget_guides.csv` (CRISPR guides with coverage, off-target count, specificity), `ltr_struct_summary.csv` (full-length vs solo LTR counts), `subfamily_tree.nwk` (a Newick tree for FigTree/iTOL), and `divergence_per_locus.csv` (per-copy divergence for the repeat-landscape plot).

Structure predictions (`reports/fold/`). If ColabFold ran, this folder holds the predicted 3-D structure of each cluster's consensus ORF: `*.pdb` (open in PyMOL, ChimeraX, or https://molstar.org/viewer), `*_plddt.png` (per-residue confidence; > 70 reliable, > 90 high), and `*_pae.png` (predicted aligned error; blue blocks = confident domains).

9.9 Following a job and the dashboard

`07_visualizations/index.html` is a dashboard linking every figure produced. `bsub_main.out` / `bsub_fold.out` are the live cluster logs, and `reports/standout_analysis.log` records each Stage 11 step with timings — check it first if a figure is missing:

```
[phylo] ... ok (612s)
[grna] ... ok (94s)
[fold] ... FAILED rc=1 (12s) <- look here if a result is absent
```

To compile the auto-generated PDF report stitching every figure and number together:

```
tectonic gameca_report.tex # or: pdflatex gameca_report.tex
```

10 Updates

Python scripts are refreshed silently from the latest commit on `main` every launch. No action required.

App binary When a new GitHub Release has a higher version, a banner appears on next launch. Click **Install update**; the new `.dmg` is downloaded, signature-verified, and installed. Relaunch when prompted.

To update manually, download the latest `.dmg` from Releases and repeat §4.

11 Troubleshooting

"GAMECA is damaged and cannot be opened." macOS quarantine. Right-click **GAMECA.app** → **Open** → **Open** (one time). If it persists, **System Settings** → **Privacy & Security** → **Open Anyway**.

Blank screen or endless spinner. The Python sidecar failed to start. Confirm you downloaded the correct architecture. Open **Console.app**, filter for "GAMECA", and look for `ImportError` / `sidecar not found`.

Dependency install fails on first launch. Run it manually, then relaunch:

```
python3 -m venv --system-site-packages ~/gameca_venv
source ~/gameca_venv/bin/activate
pip install -r \
  /Applications/GAMECA.app/Contents/Resources/requirements.txt
```

HPC connection fails. Verify SSH works from Terminal (`ssh -i ~/.ssh/id_ed25519 user@host`). For Duo clusters add a `ControlMaster` entry (see §8).

***Jobs stuck in* pend.** Normal when the queue is full or a fold job is waiting on its main job. Check slots with `bqueues normal`; inspect a specific job with `bjobs -l <JOBID>`.

GO stage produced only `GO_SKIPPED.txt`. No motifs passed the enrichment p-value threshold — expected for small or highly divergent families. Lower `-p-threshold` or inspect `enrichment_results/` directly.

Motif/TFBS folders are empty. JASPAR motif data could not be fetched (network/proxy). Check `echo $https_proxy`; GAMECA reads standard proxy env vars. A `MOTIF_ANALYSIS_FAILED.txt` marker explains why.

ColabFold hangs or crashes on HPC. Old host GCC. Use a Singularity image:

```
python run_stage11_all.py --only fold \
   --singularity-image /path/to/colabfold.sif \
   --no-gpu # omit if GPUs are available
```

Sequences are all "N" or empty. The UCSC API was unreachable from the compute node. Check the proxy as above, or supply a local genome FASTA via `-genome`.